Step 1: Create Project Folder and Virtual Environment

First, make a new directory for your DVC project. This will store all files, scripts, and data. A virtual environment helps you manage dependencies cleanly, keeping this project isolated from others.

```
mkdir iris-dvc && cd iris-dvc
python -m venv .venv
source .venv/bin/activate   # For Windows: .venv\Scripts\activate
```

Step 2: Install Required Libraries

Install DVC and essential Python libraries like pandas, scikit-learn, and joblib. These will help in building the RandomForest model and versioning data.

```
pip install dvc pandas scikit-learn joblib
```

Step 3: Initialize Git and DVC

Git tracks your code versions, and DVC tracks your data and model versions. This step creates configuration files for both tools.

```
git init
dvc init
```

Step 4: Create params.yaml File

This YAML file stores tunable parameters like test size and number of trees. DVC will use these parameters to track changes in experiments.

```
split:
  test_size: 0.2
train:
  n_estimators: 100
  random_state: 42
```

Step 5: Create Data and Model Scripts

These scripts define how your data is loaded, processed, and how your model is trained. DVC will treat each as a separate stage in the pipeline.

```
# prepare.py
from sklearn.datasets import load_iris
import pandas as pd
data = load_iris(as_frame=True)
df = pd.concat([data.data, pd.Series(data.target, name='target')], axis=1)
df.to_csv('data/iris.csv', index=False)

# train.py
import pandas as pd
import yaml
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import joblib, json, os

params = yaml.safe_load(open('params.yaml'))

os.makedirs('model', exist_ok=True)
```

```
os.makedirs('metrics', exist_ok=True)

df = pd.read_csv('data/iris.csv')
X, y = df.drop('target', axis=1), df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=params['split']['test_size'], random_state=params['train']['random_
)

model = RandomForestClassifier(
    n_estimators=params['train']['n_estimators'],
    random_state=params['train']['random_state']
)
model.fit(X_train, y_train)
preds = model.predict(X_test)

acc = accuracy_score(y_test, preds)
joblib.dump(model, 'model/model.pkl')
json.dump({'accuracy': acc}, open('metrics/eval.json', 'w'))
```

Step 6: Create dvc.yaml File

This file defines the pipeline stages and their dependencies. DVC runs them in order to produce final outputs like model and metrics.

```
stages:
  prepare:
    cmd: python prepare.py
    outs:
      - data/iris.csv

  train:
    cmd: python train.py
    deps:
      - data/iris.csv
      - train.py
    params:
      - split.test_size
      - train.n_estimators
      - train.random_state
    outs:
      - model/model.pkl
    metrics:
      - metrics/eval.json:
          cache: false
```

Step 7: Run and Test the Pipeline

Reproduce your entire workflow automatically using DVC. If any input changes, only affected stages will re-run. This ensures efficiency and reproducibility.

```
dvc repro
```

Step 8: Setup a DVC Remote

A remote location stores your datasets and models. It can be local, on a server, or cloud storage like S3. This allows sharing artifacts across machines or collaborators.

```
mkdir -p /tmp/dvc-remote
dvc remote add -d myremote /tmp/dvc-remote
dvc push
```

Step 9: Connect with GitHub and Push Code

Now you can link your local project with a GitHub repository. Push all commits to GitHub, while large data and models are pushed with DVC.

```
git add .
git commit -m "Add DVC pipeline for Iris dataset"
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Step 10: Collaborator Workflow

Anyone who clones your repo can fetch the data and re-run the pipeline easily. DVC ensures consistent experiments and data integrity.

```
git clone <your-repo-url>
cd iris-dvc
pip install -r requirements.txt
dvc pull
dvc repro
```